Block Blast RL Agent with MCTS and Neural Network

Samuel Lee
UC San Diego
hsl023@ucsd.edu

1 Introduction

Block Blast is an 8x8 grid puzzle game that is inspired by Tetris (See Figure 1). The player must choose one of three randomly chosen block forms to put on the board during each round. If a line clears when the player completely fills a row or column with blocks, the player receives points. Combo multipliers for greater scores are obtained by clearing many lines in a row. The objective is to maximize the score by continuing to place blocks and clear lines until there are no more possible moves. Block Blast may be hard even if the rules are simple. The goal of this study is to create an agent that outperforms new human players.

In this project, we want to combine Monte Carlo Tree Search with a neural network function approximator. The success of planning algorithms like AlphaGo and AlphaZero, where MCTS driven by learned policy and value networks yielded superhuman outcomes in board games, inspires ours. Our goal is to integrate a neural network into the MCTS planning process that assesses Block Blast conditions and recommends promising options. We anticipate that this hybrid strategy will better explore the game's decision tree than pure reinforcement learning, producing an agent that may get higher scores than both human players and random bots.

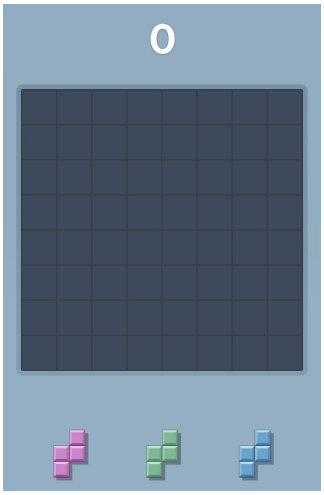

Figure 1: The Beginning State of a Single Block Blast Game

2 Related Work

Action Masking in RL In RL, handling incorrect actions in vast or dynamic action spaces is a significant problem. In their analysis of invalid action masking for policy-gradient techniques, Huang and Ontañón (2020) provide a theoretical foundation for the idea that masking out illegal movements still results in a legitimate policy gradient update (1). They have shown that masking is essential for

effective learning as the percentage of incorrect movements increases. This is especially important in environments where the number of legal actions varies per state, as in many puzzle or combinatorial games.

MCTS with Neural Networks AlphaGo and AlphaZero are groundbreaking examples of combining MCTS with deep neural networks. AlphaGo employed supervised learning and reinforcement learning to train its policy and value networks. It used MCTS to guide decision-making in the game of Go, achieving superhuman performance against top human players (2). AlphaZero took this approach further, achieving strong performances in Go, Chess, and Shogi through self-play and reinforcement learning. This demonstrated the power and flexibility of neural-guided tree search (3).

3 Methods

3.1 Problem Formulation

Block Blast can be modeled as a Markov Decision Process (MDP):

- **State** s_t: Represents the current state of the game, including the 8×8 grid configuration and the set of three available block shapes.
- **Action** a_t: The choice of which block shape to place and at which location on the grid. Only legal placements are allowed.
- **Reward** r_t: The immediate reward received after performing a_t. This is computed based on line clears and combo multipliers.
- **Transition**: Applying an action a_t transforms the environment to a new state s_{t+1}, with a new block drawn to replace the used one. Transitions are stochastic due to the random nature of block shape generation.

The goal is to maximize the cumulative reward over time, defined as the sum of all rewards:

$$\sum_{t=0}^{T} r_t \quad \text{where } T \text{ is the terminal timestep (game over).}$$

3.2 Reward Function

The environment provides a shaped reward at each timestep, designed to encourage valid, high-scoring, and strategic placements. Let:

- p_t: points gained from line clears and bonuses (e.g., combo, all-clear),
- l_t: number of lines cleared (both row and columns),
- v_t: indicator of whether the placement was valid,
- d_t: indicator for game-ending move (1 if this move causes game over, else 0).

Then the shaped reward is:

$$r_t = \begin{cases} -1.0 & \text{if } v_t = 0 \\ 0.3 + 0.1 \cdot p_t + 1.5 \cdot l_t & \text{if } v_t = 1 \text{ and } d_t = 0 \\ -5.0 & \text{if } d_t = 1 \end{cases}$$

3.3 Integrating MCTS and Neural Network

We use a neural network $f_\theta(s)$ to assist the MCTS agent. This network predicts:

- **Policy** $P(s)$: A probability distribution over all possible actions in state s, conditioned on the current board configuration and available pieces. This helps MCTS prioritize which actions to explore first.
- **Value** $V(s)$: A scalar estimating the expected cumulative reward starting from state s. This replaces the need for lengthy rollouts.

The integration with MCTS occurs in four stages:

1. **Selection:** MCTS begins at the root node (current state) and traverses the tree by selecting actions that maximize the PUCT (Predictor + Upper Confidence bound for Trees) formula:

$$Q(s,a) + c \cdot P(s,a) \cdot \frac{\sqrt{\sum_b N(s,b)}}{1 + N(s,a)}$$

   Here, $Q(s,a)$ is the current mean value of taking action a in state s, $N(s,a)$ is the visit count for that action, $P(s,a)$ is the prior probability given by the policy network, and c is an exploration constant.
2. **Expansion:** Once a leaf node is reached, all valid actions from that state are added as new child nodes. Each new child represents the result of applying one of these legal actions. For each new child state s', the neural network $f_\theta(s')$ is queried to provide $P(s')$ and $V(s')$, which are stored in the tree for later use.
3. **Evaluation:** Rather than performing a random simulation to the end of the game, we use the value $V(s')$ predicted by the neural network as an approximation of the outcome from the leaf node.
4. **Backpropagation:** The predicted value $V(s')$ is propagated back up the tree to update the statistics of each node and action along the traversal path. Specifically, the visit counts $N(s,a)$ are incremented, and the value estimates $Q(s,a)$ are updated accordingly.

Strategic planning that adapts based on cumulative experience becomes possible due to a combination of search (MCTS) and learning (neural network). In Block Blast, where specific placements might result in beneficial combo setups or unfilled gaps, it helps prevent shortsighted moves and facilitates long-term optimization.

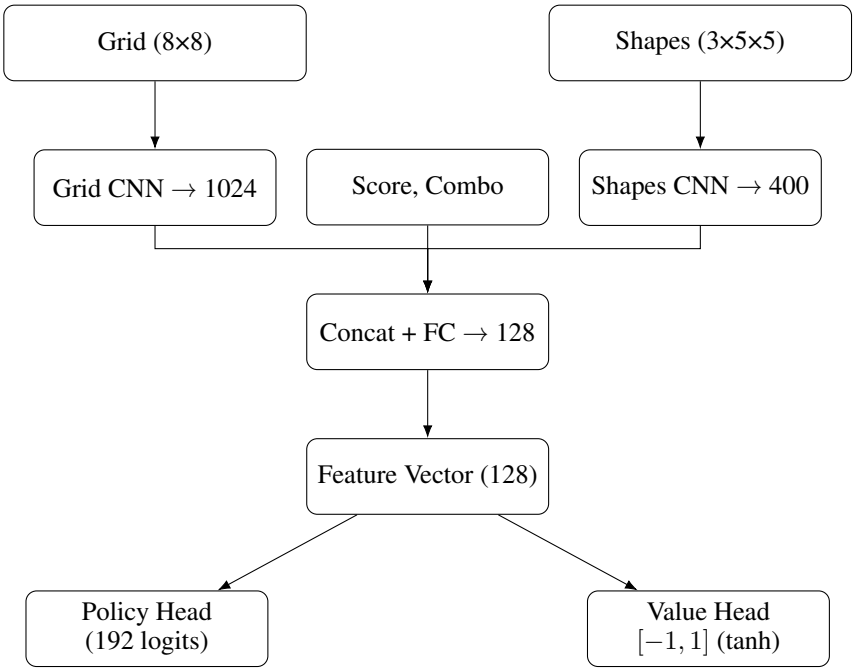

Figure 2: Architecture of the neural network used in the Block Blast agent.

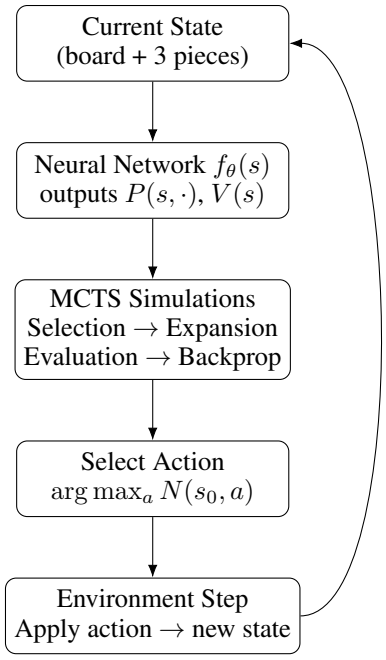

Figure 3: Overview of the MCTS + Neural Network decision loop.

3.4 MCTS Pseudocode

Algorithm 1 MCTS with Neural Network Guidance

```
function MCTS(s_0)
    for i = 1 to N simulations do
        Traverse tree from s_0 using PUCT until leaf s_L
        if s_L not terminal then
            Expand all valid a from s_L; compute P(s'), V(s')
        end if
        Backpropagate V(s') along the visited path
    end for
    return arg max_a N(s_0, a)
end function
```

4 Experiments and Results

4.1 Agents

- **Random Agent**: Chooses legal actions uniformly at random from the action mask. This serves as a baseline to assess whether an agent exhibits any intelligent behavior.
- **AlphaZero-Inspired MCTS Agent**: Our proposed agent combines Monte Carlo Tree Search with a neural network trained to predict both action priors and state values.

4.2 Experimental Setup

The MCTS agent was trained using self-play over 1000 episodes. For each move, 128 simulations were run to explore future outcomes. The network was updated every 10 episodes using 20 training steps per update. Temperature parameters controlled exploration: moves within the first 20 steps used a softmax temperature of 1.0 to encourage diversity, while later moves used 0.1 to promote exploitation of high-value actions. Rewards were scaled by a factor of 100.0 to stabilize training.

4.3 Evaluation Protocol

We evaluated both agents on 500 independent game episodes. The performance metrics are the total game score and the number of steps (moves) per episode. The MCTS agent used a fixed number of simulations per move (64) and was not allowed to exceed real-time constraints to maintain fairness.

4.4 Results

Figure 4 compares the average score and number of moves taken per game between the two agents. Figure 5 shows the complete distribution of these metrics over 500 evaluation episodes. The results clearly show that the MCTS + NN agent performs better than the random baseline.

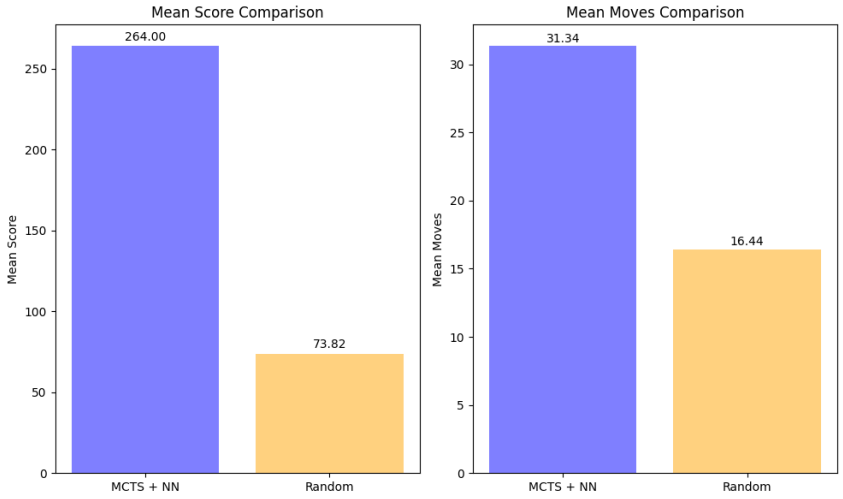

Figure 4: Mean Score and Mean Moves Comparison between Agents

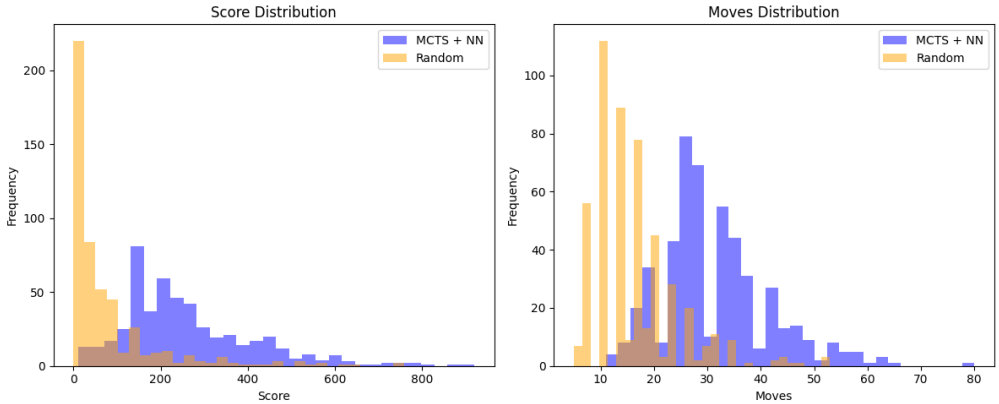

Figure 5: Distribution of Score and Moves for MCTS + NN and Random Agents

Mean Performance Comparison. The MCTS + NN agent achieved a mean score of 264.00, while the random agent scored 73.82. This means it performed approximately 3.9 times better on average. Although it took more steps per game, 31.34 compared to 16.44, the increase is only about 1.9 times. This difference shows that the MCTS agent lasts longer, and it earns significantly more reward per move than the random agent.

Distributional Insights. The histograms in Figure 5 support these findings. The random agent's scores are mostly clustered at the lower end, showing many short games with little progress. In

contrast, the MCTS agent has a wide score range with many high scores. The same trend is evident in move distributions: MCTS has more consistency and a better chance of surviving longer.

5 Conclusion

In this study, we developed an agent for Block Blast by combining Monte Carlo Tree Search (MCTS) with a neural network trained to predict policy and value functions. This mixed approach allows the agent to plan for possible future states while using learned knowledge to guide its search more effectively. Our experiments show that the MCTS + NN agent performs much better than a random baseline in both average score and number of steps survived. The agent achieves higher scores and shows greater consistency and adaptability, indicated by its wider range of outcomes. These results highlight the benefits of combining planning and learning, especially in complex puzzle environments where long-term dependencies and spatial reasoning are important.

Although the current agent already demonstrates strong performance, future enhancements could involve more complex network architectures, curriculum training, or adding reinforcement learning phases. Also, comparing it to human players or other learning-based agents would provide more context for its abilities. Overall, this project confirms the effectiveness of neural-guided tree search in single-player combinatorial settings and sets the stage for creating even stronger agents in similar decision-making challenges.

References

[1] S. Huang and S. Ontañón, *A Closer Look at Invalid Action Masking in Policy Gradient Algorithms*, The International FLAIRS Conference Proceedings, vol. 35, 2022. Available at: http://dx.doi.org/10.32473/flairs.v35i.130584

[2] D. Silver et al., *Mastering the game of Go with deep neural networks and tree search*, Nature, vol. 529, pp. 484–489, 2016. https://doi.org/10.1038/nature16961

[3] D. Silver et al., *Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm*, arXiv preprint arXiv:1712.01815, 2017. Available at: https://arxiv.org/abs/1712.01815